COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 / Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	09/01/2026	**DATE SUBMITTED:**	09/08/2026
NAME:	Lance Cristopher S. Salcedo	**INSTRUCTOR:**	Engr. Jimlord Quejardo

SEATWORK 7.2 **Algorithms Complexity**

1. PROCEDURE OUTPUT

Question	Answer	Explanation

1. Finding an Element
`for (int i = 0; i < n; i++)`
`{`
`  if (arr[i] == target)`
`  {`
`  return i;`
`  }`
`}`
	Best-Case Time Complexity:
	O(1)
	
	Worst-Case Time Complexity:
	O(n)
		Best-Case Time Complexity:
		The loop returns after the first interaction which is (arr[i] == target).
		
		Worst-Case Time Complexity:
		The loop must iterate based on how many is the n, The target is at the last index.

2. Printing All Elements
`for (int i = 0; i < n; i++)`
`{`
`  cout << arr[i] << endl;`
`}`
	Best-Case Time Complexity:
	O(n)
	
	Worst-Case Time Complexity:
	O(n)
		Best-Case Time Complexity:
		There is no return or conditions (like if-else), then the loop regardless of what must execute based on n.
		
		Worst-Case Time Complexity:
		Regardless of what the value is in the array, the loop is still going to execute based on n.

3. Nested Loops
`for (int i = 0; i < n; i++)`
`{`
`  for (int j = 0; j < n; j++)`
`  {`
`  cout << i << " " << j << endl;`
`  }`
`}`
	Best-Case Time Complexity:
	O(n^2)
	
	Worst-Case Time Complexity:
	O(n^2)
		Best-Case Time Complexity:
		There is no return or conditions (like if-else), then the loop regardless of what must execute based on n. This can also happen on int j so that is why it is O(n^2).
		
		Worst-Case Time Complexity:
		It still executes based on n (in this

		case n^2) which means it is the same as best-case.

4. Conditional Processing
`if (n > 100)`
`{`
`  for (int i = 0; i < n; i++)`
`  {`
`  cout << arr[i];`
`  }`
`}`
`else`
`{`
`  cout << "Small input";`
`}`
	Best-Case Time Complexity:
	O(1)
	
	Worst-Case Time Complexity:
	O(n)
		Best-Case Time Complexity:
		No loop was executed here so it goes to the else regardless of what the value is n.
		
		Worst-Case Time Complexity:
		It becomes true so it won't go to else and it loops directly to n.

5. Fixed Inner Loop
`for (int i = 0; i < n; i++)`
`{`
`  for (int j = 0; j < 10; j++)`
`  {`
`  cout << i << j;`
`  }`
`}`
	Best-Case Time Complexity:
	O(n^2)
	
	Worst-Case Time Complexity:
	O(n^2)
		Best-Case Time Complexity:
		There is no return or conditions (like if-else), then the loop regardless of what must execute based on n. This can also happen on int j so that is why it is O(n^2).
		
		Worst-Case Time Complexity:
		It still executes based on n (in this case n^2) which means it is the same as best-case.

6. Two Consecutive Loops
`for (int i = 0; i < n; i++)`
`{`
`  cout << arr[i];`
`}`
 	
`for (int j = 0; j < n; j++)`
`{`
`  cout << arr[j] * 2;`
`}`
	Best-Case Time Complexity:
	O(n)
	
	Worst-Case Time Complexity:
	O(n)
		Best-Case Time Complexity:
		Because they are separate just like the other two loops, there is no return or conditions, then the loop still executes based on n.
		
		Worst-Case Time Complexity:
		Even though they are separate it still executes based on n (even with two separate loops) which means it is the same as best-case.

7. Dividing by Two
`int x = n;`
`while (x > 1)`
`{`
`x = x / 2;`
`}`
	Best-Case Time Complexity:
	O(log n) can be also O(1)
	
	Worst-Case Time Complexity:
	O(log n)
		Best-Case Time Complexity:
		Since the value of x is divided by 2 in each iteration, thus reducing n down to 1.
		
		Worst-Case Time Complexity:
		It repeatedly reduces halves to the value of x.

8. Nested Loop With Different Limits
`for (int i = 0; i < n; i++)`
`{`
`for (int j = 0; j < i; j++)`
`{`
`cout << i << j;`
`}`
`}`
	Best-Case Time Complexity:
	O(n^2)
	
	Worst-Case Time Complexity:
	O(n^2)
		Best-Case Time Complexity:
		There is no return or conditions (like if-else), then the inner loop is dependent on i for the outer loop (j > i). Also it executes based on n^2 - n/2.
		
		Worst-Case Time Complexity:
		It still executes based on n(n-1)/2.

9. Early Termination
`for (int i = 0; i < n; i++)`
`{`
`if (arr[i] == target)`
`{`
`cout << "Found";`
`break;`
`}`
`}`
	Best-Case Time Complexity:
	O(1)
	
	Worst-Case Time Complexity:
	O(n)
		Best-Case Time Complexity:
		The loop breaks after the first interaction which is (arr[i] == target), it will execute regardless of n.
		
		Worst-Case Time Complexity:
		The loop must execute based on how many is the n times every element in the array. The target is at the last.

10. Two-Level Search
`for (int i = 0; i < n; i++)`
`{`
`for (int j = 0; j < n; j++)`
`{`
`if (arr[i] == target)`
`{`
`break;`
`}`
`}`
`}`
	Best-Case Time Complexity:
	O(n)
	
	Worst-Case Time Complexity:
	O(n^2)
		Best-Case Time Complexity:
		If the arr[i] == target is true for every element in the array and for each n that iterates in the outer loop which means that the inner loop (arr[i] == target) is also true. It also executes n times, the inner loop executes 1 per iteration based on the outer loop which means n x 1 = n and also has a condition which is break.
		
		Worst-Case Time Complexity:
		If every scenario is false, which

		means the if statement didn't trigger which doesn't trigger the break condition too. Both the inner and outer loops have to run n iterations, executing it n^2 times.

11. Linear Loop With Logarithmic Inner Work
`for (int i = 0; i < n; i++)`
`{`
`  int x = n;`
`  while (x > 1)`
`  {`
`  x = x / 2;`
`  }`
`}`
	Best-Case Time Complexity:
	O(n log n)
	
	Worst-Case Time Complexity:
	O(n log n)
		Best-Case Time Complexity:
		The outer loop executes n times. There are also no conditions. The outer loop, x is reset to n, the inner loop repeatedly divides to 2.
		
		Worst-Case Time Complexity:
		Same as best-case but the outer loop runs n times and the inner loop is consistently executes log n times for each iteration.

12. Conditional Nested Loop
`if (arr[0] == target)`
`{`
`  cout << "Found";`
`}`
`else`
`{`
`  for (int i = 0; i < n; i++)`
`  {`
`  for (int j = 0; j < n; j++)`
`  {`
`  cout << arr[i] + arr[j];`
`  }`
`  }`
`}`
	Best-Case Time Complexity:
	O(1)
	
	Worst-Case Time Complexity:
	O(n^2)
		Best-Case Time Complexity:
		When arr[0] == target becomes true, there are no loops but it prints. It executes to O(1) regardless of the array size n.
		
		Worst-Case Time Complexity:
		When arr[0] == target becomes false, which means it will do the else statement. The outer loop runs n times for each iteration and the inner loop also runs n times, which means n x n = n^2.

13. Finding the Maximum
`int max = arr[0];`
`for (int i = 1; i < n; i++)`
`{`
`  if (arr[i] > max)`
`  {`
`  max = arr[i];`
`  }`
`}`
	Best-Case Time Complexity:
	O(n)
	
	Worst-Case Time Complexity:
	O(n)
		Best-Case Time Complexity:
		It's an unsorted array which means it will be inspected at least once. The loop must complete n-1 iterations.
		
		Worst-Case Time Complexity:
		When arr[i] > max becomes true for every single iteration, the loop still runs for n-1 times.

14. Three Nested Loops
`for (int i = 0; i < n; i++)`
`{`
`  for (int j = 0; j < n; j++)`
`  {`
`  for (int k = 0; k < n; k++)`
`  {`
`  cout << i + j + k;`
`  }`
`  }`
`}`
	Best-Case Time Complexity:
	O(n^3)
	
	Worst-Case Time Complexity:
	O(n^3)
		Best-Case Time Complexity:
		There is no return or conditions (like if-else), then the loop regardless of what must execute based on n. This can also happen on the other two for loops thus (n^3).
		
		Worst-Case Time Complexity:
		It still executes based on n (in this case n^3) which means it is the same as best-case.

15. Comprehensive Analysis
`for (int i = 0; i < n; i++)`
`{`
`  if (arr[i] == target)`
`  {`
`  return i;`
`  }`
 	
`  int x = n;`
 	
`  while (x > 1)`
`  {`
`  x = x / 2;`
`  }`
`}`
	Best-Case Time Complexity:
	O(1)
	
	Worst-Case Time Complexity:
	O(n log n)
		Best-Case Time Complexity:
		It becomes true and also has a condition which is return i. It only has a single comparison because of it.
		
		Worst-Case Time Complexity:
		It becomes false which means it would go to the while statement. The array is either on the last or not at all. For each n in the outer loop, the inner loop iterates it by dividing n by 2. Thus, n log n.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

N/A

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I only use Gemini for the explanation on why it happened in both best-case and**

worst-case to explain it further and any of the sentences that I made in the explanation is mine only.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.